IN5340/IN9340 Project II: Estimation Theory

Theodor Wålberg

University of Oslo

February 2026

Code: `github.com/Twaal/Statistical_Signal_Processing_IN5340/tree/master/Project_2`

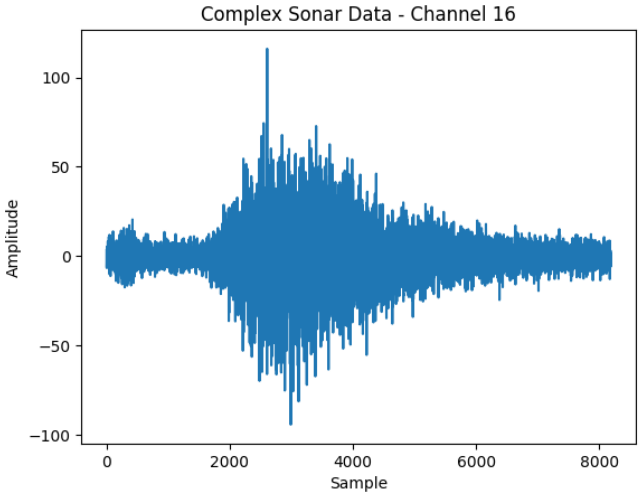
Agenda

- Introduction and dataset
- 1A–1D: Maximum likelihood time-delay estimation
- 2A: Cramér–Rao lower bound
- 3A–3B: Least squares delay estimation and comparison
- Summary

Project scope and data

- Goal: estimate time delay of strongest reflector in sonar data (32 channels).
- Dataset: `sonardata2.mat`, complex matrix data $\in \mathbb{C}^{8192\times 32}$.
- Known parameters from assignment/code: $B = 30\,\text{kHz}$, $f_c = 100\,\text{kHz}$, $f_s = 40\,\text{kHz}$, $T_p = 8\,\text{ms}$.
- Methods covered: ML (matched filter), CRLB, and LSE fine delay estimation.

Raw channel example



1A: ML estimator derivation

Assume

$$x(t) = s(t - T_0) + w(t), \qquad x[n] = s(n\Delta - \tau_0) + w[n],\ w[n] \sim \mathcal{N}(0, \sigma_w^2).$$

For fixed τ_0, maximizing likelihood is equivalent to minimizing

$$J(\tau_0) = \sum_{n=0}^{N-1} \left(x[n] - s(n\Delta - \tau_0)\right)^2.$$

Expanding $J(\tau_0)$ gives ML as cross-correlation maximization:

$$\hat{\tau}_{\mathrm{ML}} = \arg\max_{\tau_0} \sum_{n=0}^{N-1} x[n] s(n\Delta - \tau_0).$$

Interpretation: delay is chosen at matched-filter peak.

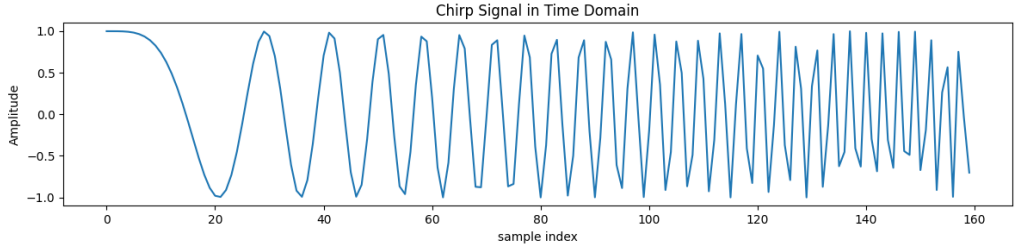
1B: Implementing chirp and matched filter

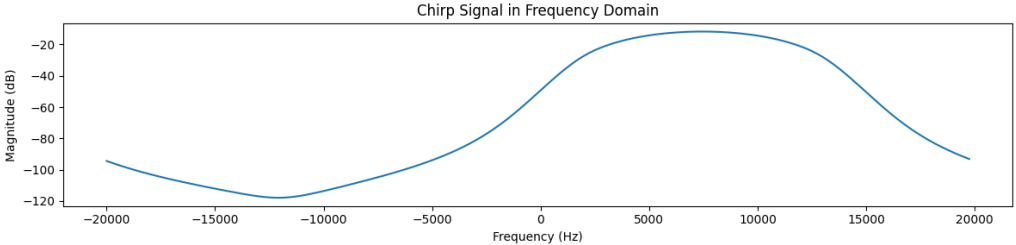
```
# chirp
B = 30000; fs = 40000; tp = 0.008
t = np.arange(0, tp/2, 1/fs)
a = B / tp
chirp = np.exp(1j * (2*np.pi) * a * t**2 / 2)

# matched filter for channel 16
x = data[:, 15]
matched_filter_output = correlate(x, chirp, mode='full')
```

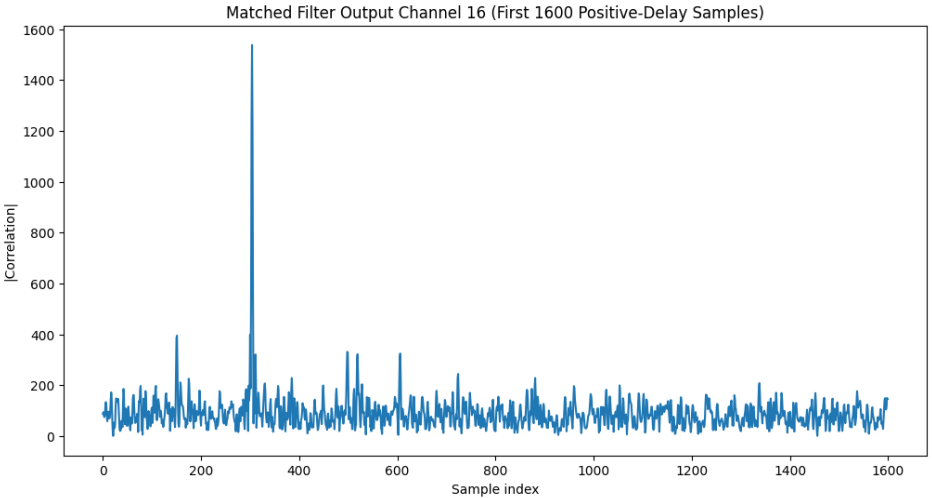
Positive-delay part is used.

1B: Chirp signal (time and frequency)





1B: Matched filter output, channel 16



Notebook print:

correlation length = 8351, original signal length = 8192.

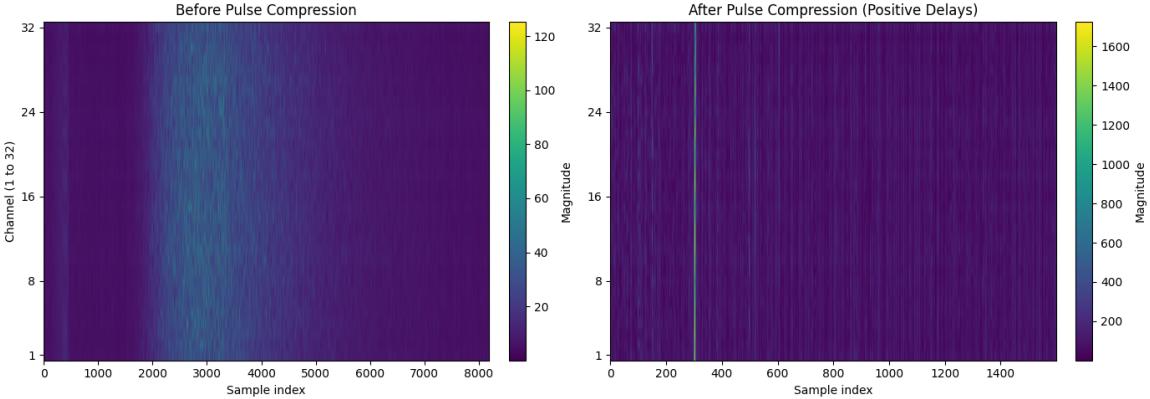
1C: Pulse compression on all channels

```
pc_list = []
for ch in range(num_channels):
    y_full = correlate(data[:, ch], chirp, mode='full')
    pc_list.append(y_full)
pulse_compressed_full = np.column_stack(pc_list)

start_idx = len(chirp) - 1
pulse_compressed = pulse_compressed_full[start_idx:, :]

before_mag_all = np.abs(data[:1600, :]).T
after_mag_all  = np.abs(pulse_compressed[:1600, :]).T
```

1C: Before vs after pulse compression



1C: Discussion

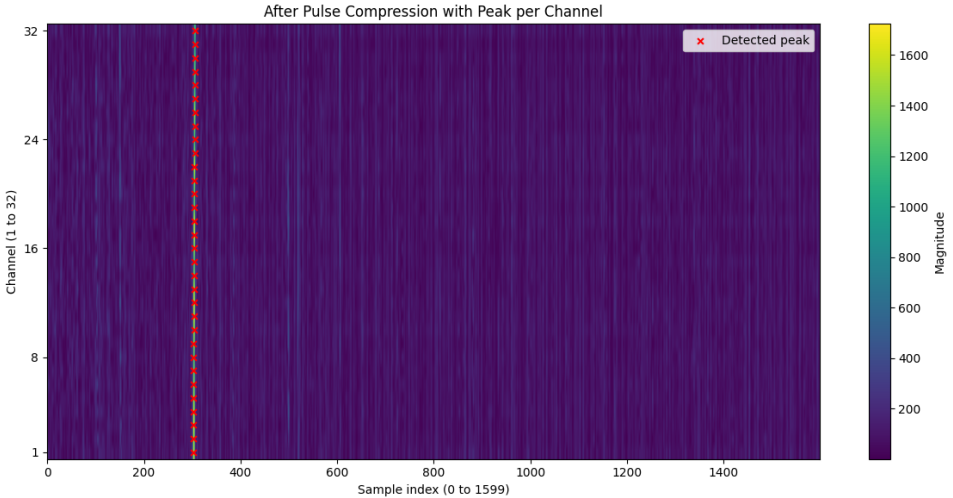
- Before compression: energy is spread in time and appears more noise-like.
- After matched filtering: peaks are sharper and stronger.
- Detectability and time-delay resolution improve.
- Output remains complex because both received data and chirp reference are complex.

1D: Peak detection per channel

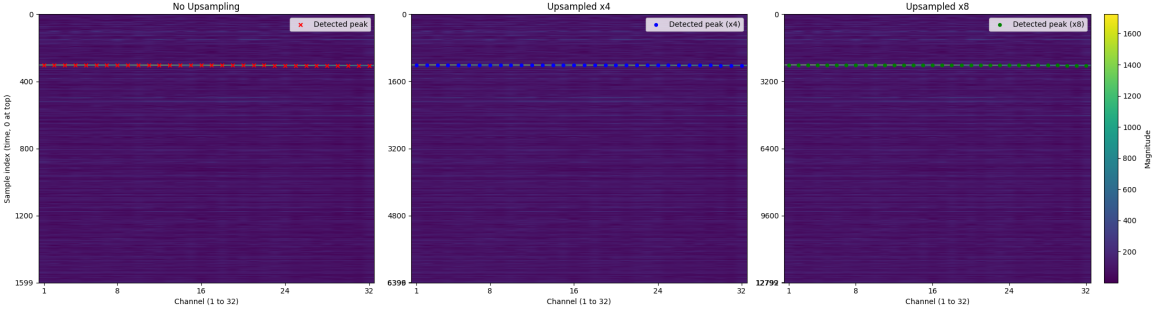
```
peak_indices = np.argmax(after_mag_all, axis=1)

after_mag_all_upsampled_4 = resample(after_mag_all,
                                     after_mag_all.shape[1]*4, axis=1)
after_mag_all_upsampled_8 = resample(after_mag_all,
                                     after_mag_all.shape[1]*8, axis=1)
peak_indices_upsampled[4] = np.argmax(after_mag_all_upsampled_4, axis=1)
peak_indices_upsampled[8] = np.argmax(after_mag_all_upsampled_8, axis=1)
```

1D: Peak per channel, no upsampling



1D: Delay estimates with upsampling x4 and x8



1D: Interpretation

- Delays are very similar across channels, with small channel-dependent differences.
- Main peak location is around sample index ≈ 304 in the 1600-sample window.
- After upsampling, index values scale with factor, while physical delay stays consistent.

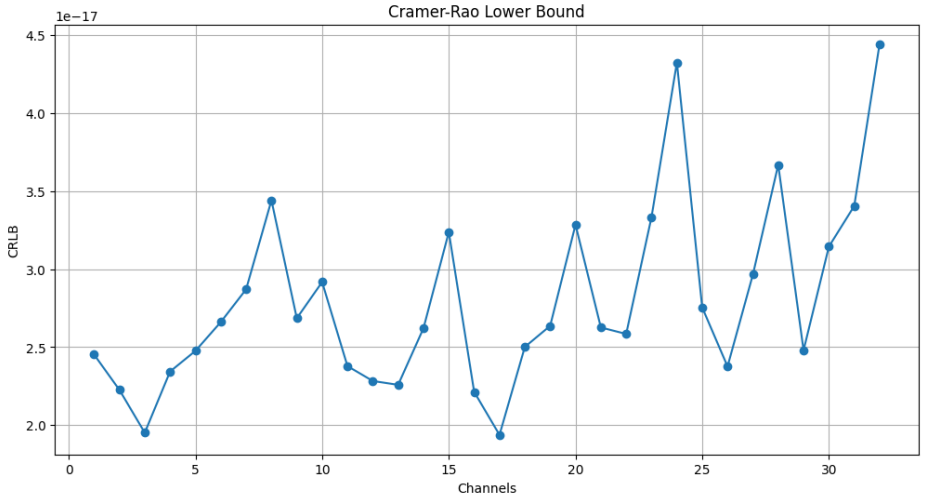
2A: SNR and CRLB computation

```
peak_magnitudes = after_mag_all[np.arange(after_mag_all.shape[0]), peak_indices]
peak_intensities = peak_magnitudes**2
noise_intensities = np.mean(np.delete(after_mag_all**2, peak_indices, axis=1), axis=1)
snr = peak_intensities / noise_intensities

b2_rms = (2*np.pi)**2 * ((fc**2) + (B**2)/12)
crlb = 1 / (2 * B * tp * snr * b2_rms)
```

Notebook print: RMS bandwidth approximation $\beta_{\text{rms}}^2 \approx 3.977 \times 10^{11}$.

2A: CRLB across channels



2A: Interpretation

- CRLB values are not identical for all channels.
- They remain in the same order of magnitude.
- Differences are expected due to finite samples and channel-dependent noise/reflector levels.

3A: LSE procedure

For pulse-compressed intensity $I[n] = |x_{pc}[n]|^2$ in the first 1600 samples:

1. Upsample with factor 4 or 8.
2. Locate dominant peak index.
3. Keep samples around peak above half-power threshold.
4. Form $y = \ln(I)$.
5. Build observation matrix $H = [1,\ t,\ t^2]$.
6. Solve LS for $\ln(I(t)) \approx a + bt + ct^2$ and get fine delay from vertex.

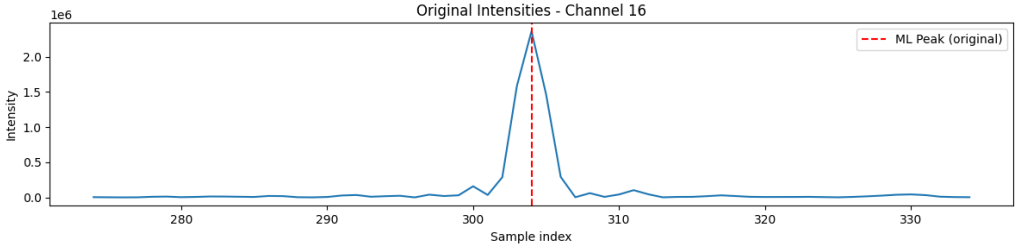
3A: LS model and fine-delay formula

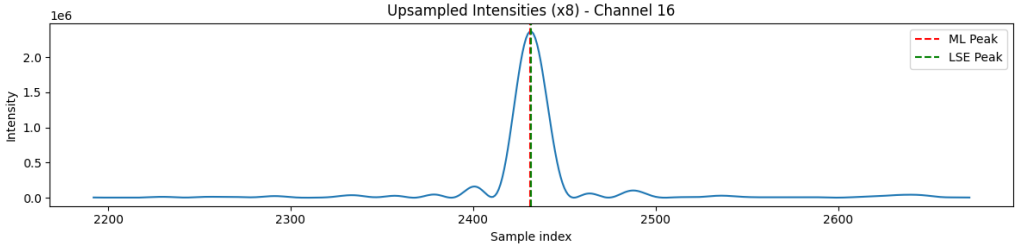
```
y = np.log(upsampled_intensities[ch, selected_indices])
t_selected = selected_indices * dt
H = np.column_stack([np.ones(len(t_selected)), t_selected, t_selected**2])
a_hat, b_hat, c_hat = np.linalg.lstsq(H, y, rcond=None)[0]
t_peak = -b_hat / (2 * c_hat)
```

Model:

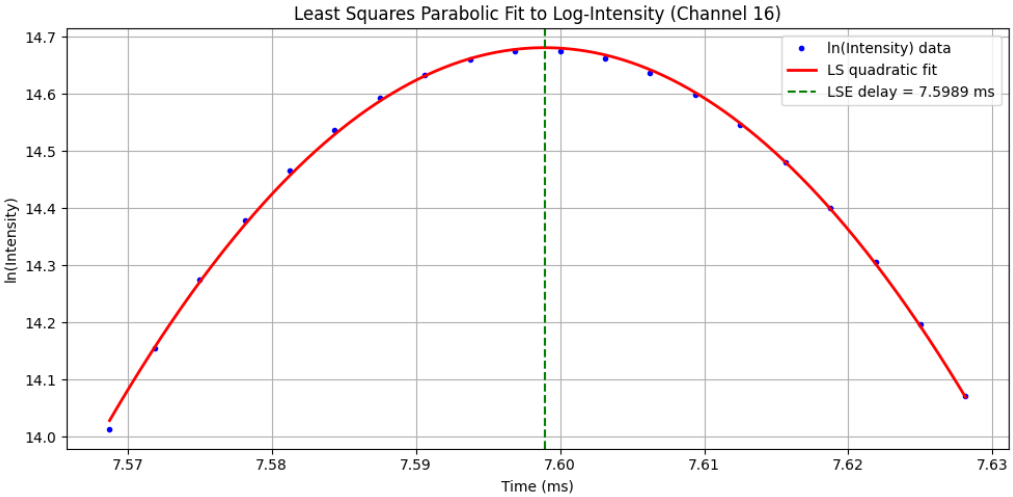
$$\ln I(t) = a + bt + ct^2, \qquad \hat{\tau}_{\mathrm{LSE}} = -\frac{\hat{b}}{2\hat{c}}.$$

3A: Original vs upsampled intensity around peak, channel 16

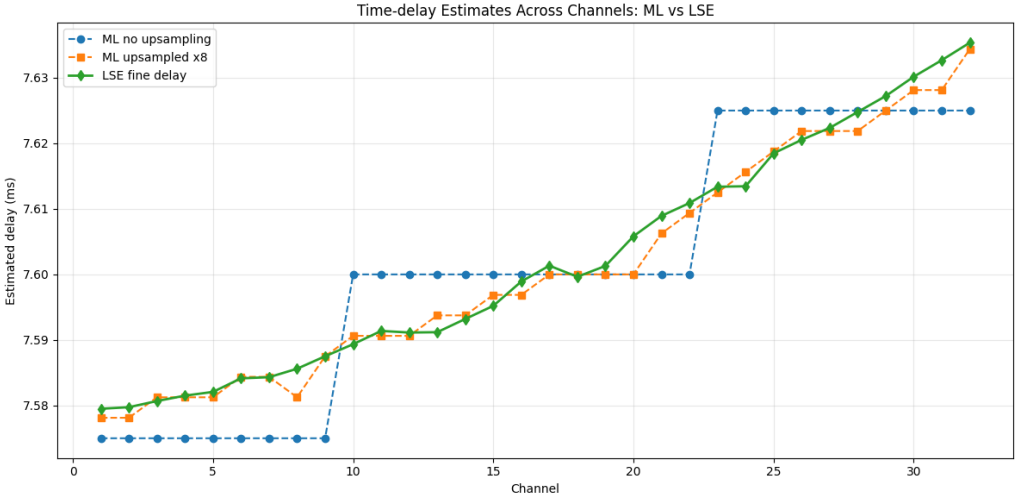




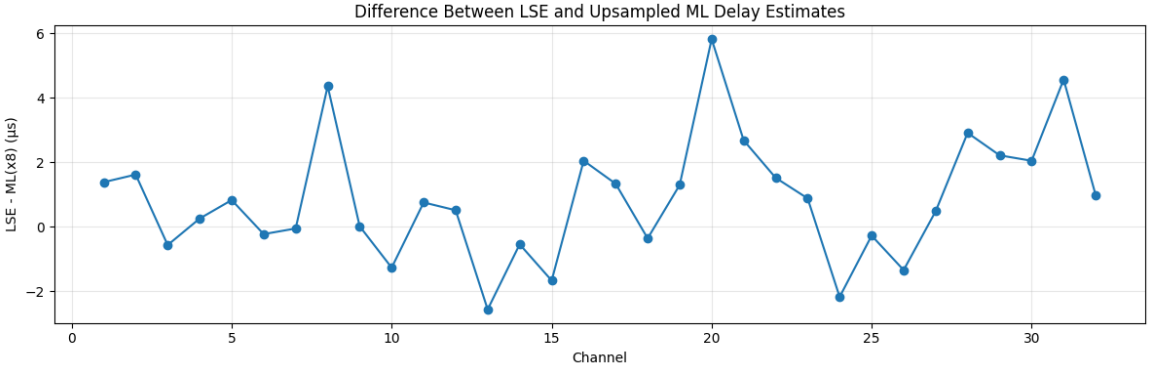
3A: Log-intensity and quadratic fit



3B: Delay estimates across channels, ML vs LSE



3B: Difference between LSE and upsampled ML



3B: Discussion

- Delays are close across channels, but not exactly identical.
- LSE gives smoother sub-sample continuous estimates.
- ML argmax is constrained to the sample grid after optional upsampling.
- LSE and ML track the same reflector, with small residual differences due to noise and local pulse-shape variation.

Summary

- Exercise 1A–1D: Derived and implemented ML delay estimation via matched filtering; validated peak consistency across channels and upsampling factors.
- Exercise 2A: Computed CRLB per channel from estimated SNR; bounds are similar but channel-dependent.
- Exercise 3A–3B: Implemented LSE fine delay from log-intensity quadratic fit; obtained sub-sample delay refinement and compared against ML.
- All figures in this deck are exported directly from executed `project_2.ipynb` outputs.